

W9D1 - (parte 1) esempi di netcat

Oggi vedremo netcat, uno strumento potente e versatile utilizzato in diversi contesti, inclusa la sicurezza informatica. Conosciuto anche come "TCP/IP Swiss Army Knife"

Netcat offre una serie di funzionalità che consentono di comunicare attraverso TCP/IP in modo semplice e flessibile, e può essere utilizzato per creare connessioni tra due sistemi, permettendo la comunicazione bidirezionale. Ciò può essere fatto per diversi scopi, come ad esempio l'esecuzione remota di comandi, l'invio di files o per "dialogare" con processi attivi su una macchina remota.

L'originale netcat (come recita il man) è stato scritto da una persona che conosciamo come lo Hobbit e pubblicato il 28/10/1995 sulla mailing list Bugtraq e ancora visibile al seguente [indirizzo](#). Lo sviluppo dell'originale netcat si è fermato alla versione 1.10 nel 1995, e da allora esistono diversi fork più o meno simili all'originale, che implementano diverse funzionalità aggiuntive.

AUTHOR

```
This manual page was written by Joey Hess <joeyh@debian.org> and Robert Woodcock <rcw@debian.org>, cribbing heavily from Netcat's README file.
```

```
Netcat was written by a guy we know as the Hobbit <hobbit@avian.org>.
```

Uso base di netcat

Netcat può essere utilizzato come dicevamo per stabilire una connessione bidirezionale tra 2 computer, quindi ci sarà una macchina che si metterà in "listen" su una determinata porta, e l'altra macchina che si connetterà per dialogare su quella stessa porta.

Il comando per creare una connessione in ascolto è:

```
1 nc \ # il comando netcat
2 -l \ # ci mettiamo in ascolto
3 -v \ # esecuzione verbosa per avere più output
4 -p 1234 # la porta su cui ascoltare il traffico
```

Mentre per aprire un client basterà specificare l'indirizzo IP e la porta sulla riga di comando:

```
1 nc 192.168.50.100 1234
```

Negli screenshot seguenti vediamo 2 macchine che dialogano tramite netcat sulla porta 1234, il "server" in ascolto a sinistra e il client a destra:

```
File Actions Edit View Help
(kali@kali)-[~]
$ nc -lvp 1234
listening on [any] 1234 ...
192.168.50.100: inverse host lookup failed: Host name lookup failure
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 56838
ciao
ciao sono il server
█
```

```
kali@kali: ~
File Actions Edit View Help
(kali@kali)-[~]
$ nc 192.168.50.100 1234
ciao
ciao sono il server
█
```

Esecuzione di comandi remoti

Con netcat possiamo eseguire comandi in remoto il cui output ci viene mostrato sulla macchina client, utile per capire con che sistema abbiamo a che fare, o anche per avviare una shell rudimentale

Ad esempio, possiamo lanciare un'istanza di netcat che eseguirà il comando `uname -a` e ne redirigerà l'output sulla prima macchina che si collegherà alla porta **1234**. La sintassi è la seguente:

```
1 | nc -lvp 1234 -c "uname -a"
```

```
(kali@kali)-[~]
$ nc -lvp 1234 -c "uname -a"
listening on [any] 1234 ...
192.168.50.100: inverse host lookup failed: Host name lookup failure
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 47968
(kali@kali)-[~]
$
```

```
(kali@kali)-[~]
$ nc 192.168.50.100 1234
Linux kali 6.12.38+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12) x86_64 GNU/Linux
(kali@kali)-[~]
$
```

Come si vede nelle immagini sopra, l'output di `uname -a` viene visualizzato nell'istanza che si connette alla porta **1234** del nostro netcat iniziale.

Possiamo anche lanciare una shell come `/bin/bash` in ascolto ed eseguire comandi da un'istanza connessa. In questo caso la sintassi è:

```
1 | nc -lvp 1234 -e /bin/bash
```

```
File Actions Edit View Help
(kali@kali)-[~]
$ nc -lvp 1234 -e /bin/bash
listening on [any] 1234 ...
192.168.50.100: inverse host lookup failed: Host name lookup failure
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 58426

```

```
(kali@kali)-[~]
$ nc 192.168.50.100 1234
ls
bin
bin
Desktop
Documents
dos.py
Downloads
flag.txt
hs_err_pid12829.log
hs_err_pid13225.log
hs_err_pid1720.log
hs_err_pid3341.log
msfadmin@192.168.50.101
Music
Pictures
Public
Templates
Videos
worknotes.txt

```

Come vediamo, nello screenshot a destra abbiamo lanciato il comando `ls`, ricevuto dalla shell in ascolto, la quale ci ha risposto con l'output immediatamente sotto, listandoci le varie cartelle e files presenti.

Altri comandi utili da lanciare tramite netcat potrebbero essere `w`, `who`, `ip a` o `ps aux`, come vediamo negli esempi seguenti, sempre con l'istanza in listen a sinistra, e l'output a destra.

"ps axu"

```
(kali@kali)-[~]
$ nc -lvp 1234 -c "ps axu"
listening on [any] 1234 ...
192.168.50.100: inverse host lookup failed: Host name lookup failure
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 38538
(kali@kali)-[~]
$
```

```
(kali@kali)-[~]
$ nc 192.168.50.100 1234
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.3 23520 14288 ?        Ss   11:33   0:00 /sbin/init splash
root         2  0.0  0.0      0  0 ?        S    11:33   0:00 [kthreadd]
root         3  0.0  0.0      0  0 ?        S    11:33   0:00 [pool_workqueue_release]
root         4  0.0  0.0      0  0 ?        I<   11:33   0:00 [kworker/R-kvfree_rcu_reclaim]
]
root         5  0.0  0.0      0  0 ?        I<   11:33   0:00 [kworker/R-rcu_gp]
root         6  0.0  0.0      0  0 ?        I<   11:33   0:00 [kworker/R-sync_wq]
root         7  0.0  0.0      0  0 ?        I<   11:33   0:00 [kworker/R-slub_flushwq]
root         8  0.0  0.0      0  0 ?        I<   11:33   0:00 [kworker/R-netns]
root        11  0.0  0.0      0  0 ?        I<   11:33   0:00 [kworker/0:0H-events_highpri]
root        12  0.0  0.0      0  0 ?        I    11:33   0:00 [kworker/u8:0-flush-8:0]
root        13  0.0  0.0      0  0 ?        I<   11:33   0:00 [kworker/R-mm_percpu_wq]
root        14  0.0  0.0      0  0 ?        I    11:33   0:00 [rcu_tasks_kthread]
root        15  0.0  0.0      0  0 ?        I    11:33   0:00 [rcu_tasks_rude_kthread]
root        16  0.0  0.0      0  0 ?        I    11:33   0:00 [rcu_tasks_trace_kthread]
root        17  0.0  0.0      0  0 ?        S    11:33   0:00 [ksoftirqd/0]
root        18  0.0  0.0      0  0 ?        I    11:33   0:00 [rcu_preempt]
root        19  0.0  0.0      0  0 ?        S    11:33   0:00 [rcu_exp_par_gp_kthread_worke
r/0]
```

"w", "who", "ip a"

```
(kali@kali)-[~]
$ nc -lvp 1234 -c who
listening on [any] 1234 ...
192.168.50.100: inverse host lookup failed: Host name lookup failure
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 50426
(kali@kali)-[~]
$ nc -lvp 1234 -c w
listening on [any] 1234 ...
192.168.50.100: inverse host lookup failed: Host name lookup failure
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 42564
(kali@kali)-[~]
$ nc -lvp 1234 -c "ip a"
listening on [any] 1234 ...
192.168.50.100: inverse host lookup failed: Host name lookup failure
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 50536
(kali@kali)-[~]
$
```

```
(kali@kali)-[~]
$ nc 192.168.50.100 1234
kali      seat0      2025-09-06 11:33 (:0)

(kali@kali)-[~]
$ nc 192.168.50.100 1234
11:58:39 up 25 min, 1 user, load average: 0.00, 0.02, 0.00
USER      TTY      FROM          LOGIN@  IDLE   JCPU   PCPU   WHAT
kali      -                11:33    IDLE   0.00s  ?      lightdm --session-child 13 24

(kali@kali)-[~]
$ nc 192.168.50.100 1234
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
        inet 127.0.0.1/8 scope host lo
            valid_lft forever preferred_lft forever
        inet6 ::1/128 scope host noprefixroute
            valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1
    000
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
        inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0
            valid_lft forever preferred_lft forever
        inet6 fe80::89c3:47eb:da38:42e0/64 scope link noprefixroute
            valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1
    000
    link/ether 08:00:27:98:9a:fd brd ff:ff:ff:ff:ff:ff

(kali@kali)-[~]
$
```

Invio di file

Un'altra importante funzione di netcat è quella di poter inviare files tra due macchine. In maniera analoga al comando `cat` possiamo, tramite l'uso di una pipe `|`, inoltrare l'output di un comando ad una macchina che si connetta alla nostra istanza di netcat in ascolto. In questo caso la sintassi è la seguente:

```
1 | cat nomefile.ext | nc -lvp 42069
```

In questo modo abbiamo utilizzato `cat` per "leggere" il file `nomefile.ext` e mandare l'output in pipe a netcat, che a sua volta lo invierà a chiunque si connetta alla porta 42069 sulla nostra macchina.

Per ricevere il file bisognerà redirigere però l'output del netcat "cliente" su un file in modo che venga scritto su disco, con la seguente sintassi:

```
1 | nc 192.168.50.101 42069 > nomefile.ext
```

un esempio di questa funzionalità è visibile negli screenshot seguenti:

```
(kali@kali)-[~]
$ ls -l kali.png
-rw-r--r-- 1 kali kali 2405474 Feb 12 2025 kali.png
(kali@kali)-[~]
$ cat kali.png | nc -lp 42069
File System
```

```
msfadmin@metasploitable:~$ nc 192.168.50.100 42069 > bg.png
msfadmin@metasploitable:~$ ls -l bg.png
-rw-r--r-- 1 msfadmin msfadmin 2405474 2025-09-08 06:31 bg.png
msfadmin@metasploitable:~$
```

In questo esempio abbiamo preso l'immagine png "kali.png" di dimensione **2405474 bytes** e inviata alla macchina a destra, dove l'abbiamo salvata come "bg.png" e come possiamo vedere, la dimensione è la medesima.

Note

L'unica particolarità di questo utilizzo di netcat, è che non abbiamo modo di controllare il progresso dell'invio del file, quindi non possiamo sapere quando questo sia completato, in quanto netcat rimane in ascolto indefinitamente. Dopo un ragionevole lasso di tempo possiamo interrompere l'istanza "cliente" con  e avremo disponibile il file sulla nostra macchina.

Quelli visti oggi sono solo alcuni esempi di utilizzo di netcat, un vero e proprio coltellino svizzero per tutti i professionisti IT, che può rivelarsi molto utile anche in ambito cybersecurity.

Alla prossima.

